Introduction à Git

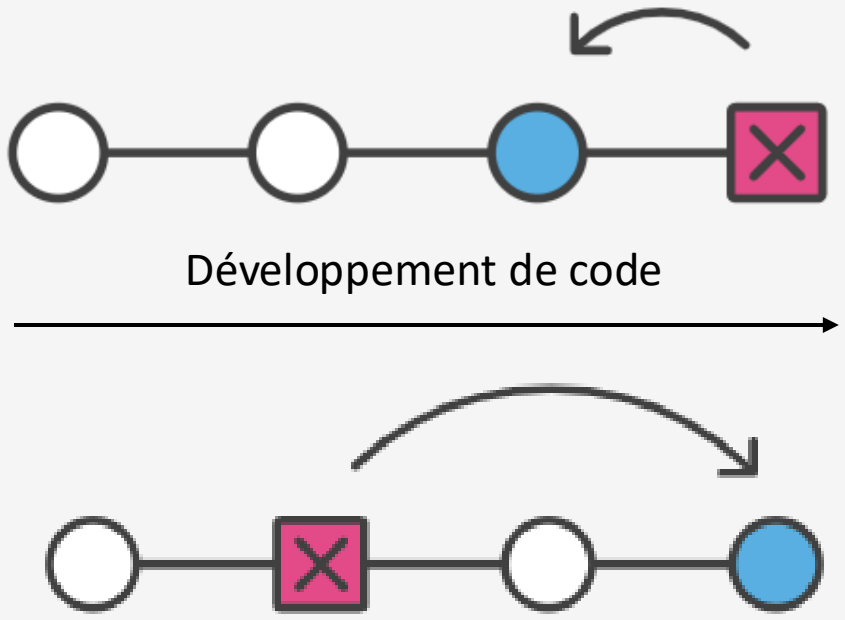
PHQ404
16 Janvier 2026
Inspiré de la présentation
de Samuel Ouellet

Survol

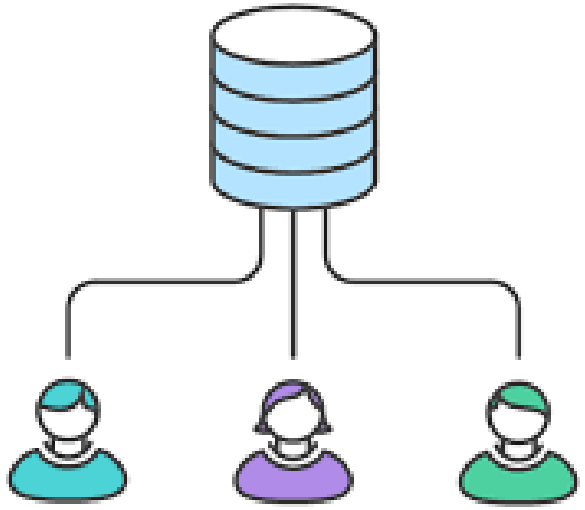
- Qu’est-ce que Git?
- Concepts nécessaires à l’utilisation de Git
- Exemples d’outils pour travailler avec Git sur votre ordinateur
- Qu’est-ce qui se trouve sur un serveur Git?
- Comment utiliser Git au quotidien?
- Introduction aux outils qu’offrent les serveurs Git pour les développeurs

Qu'est-ce que Git?

- Git est un logiciel de gestion de versions décentralisé.
- Conservation de l'historique de l'évolution de fichiers.
- Collaboration en parallèle vers un emplacement centralisé.



Git—Undoing changes | Atlassian Git Tutorial. (s. d.). Consulté 17 mai 2021, à l'adresse https://www.atlassian.com/git/tutorials/undoing-changes



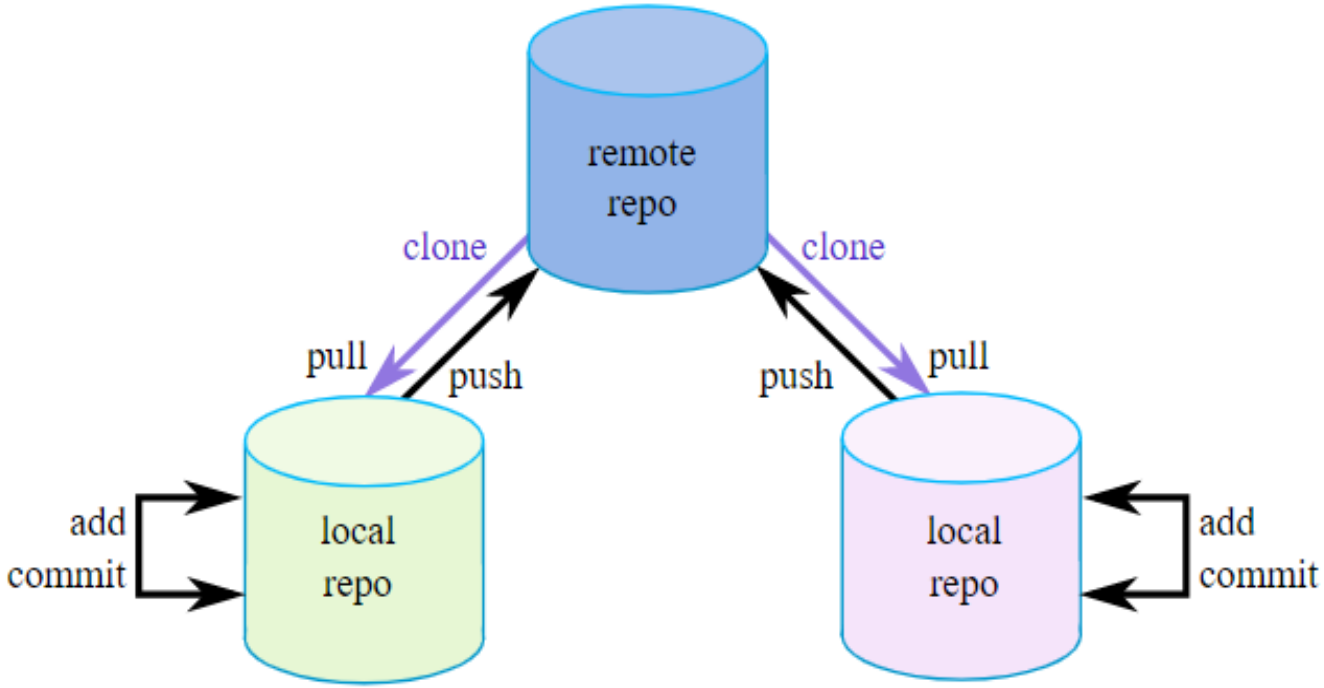
Technical Foundations of Informatics. (s. d.). Consulté 17 mai 2021, à l'adresse https://info201-s17.github.io/book/git-collaboration.html

Quels sont les concepts importants à savoir ?

- La nuance entre le répertoire local et le répertoire sur le serveur.
- Commit.
- Push et Pull.
- Branch.
- Merge.
- Fork.
- Pull Request (PR)

La nuance entre le répertoire local et le répertoire sur le serveur

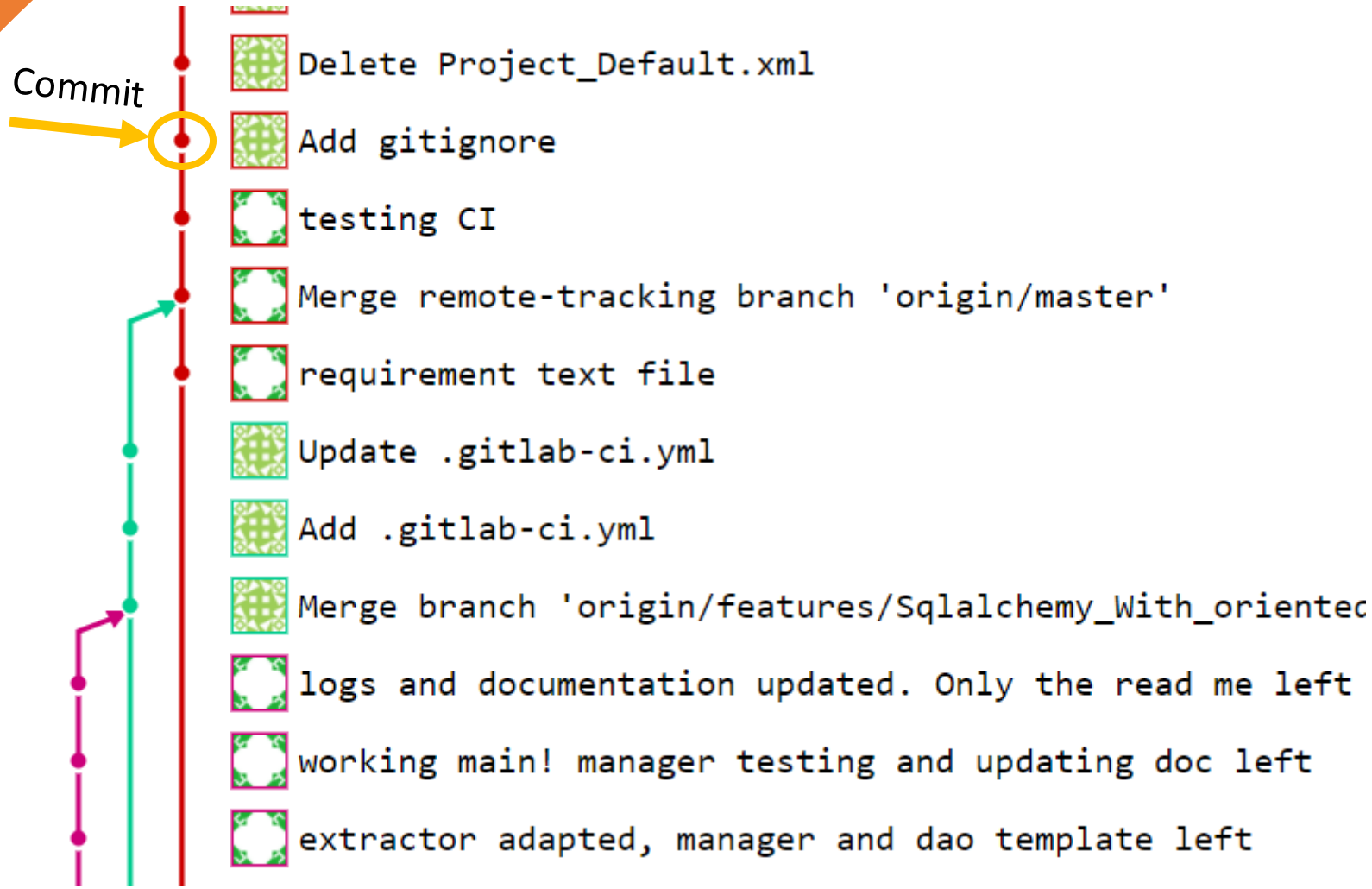
- Le clonage permet de créer le répertoire local associé au répertoire en ligne.
- Les modifications faites sur un répertoire ou sur l'autre ne se transfèrent pas automatiquement. Il faut donc s'assurer de constamment propager l'information.
- Ce concept est particulièrement important lors d'une collaboration.



Git. (s. d.). Consulté 17 mai 2021, à l'adresse https://www.cs.swarthmore.edu/~newhall/unixhelp/git.php

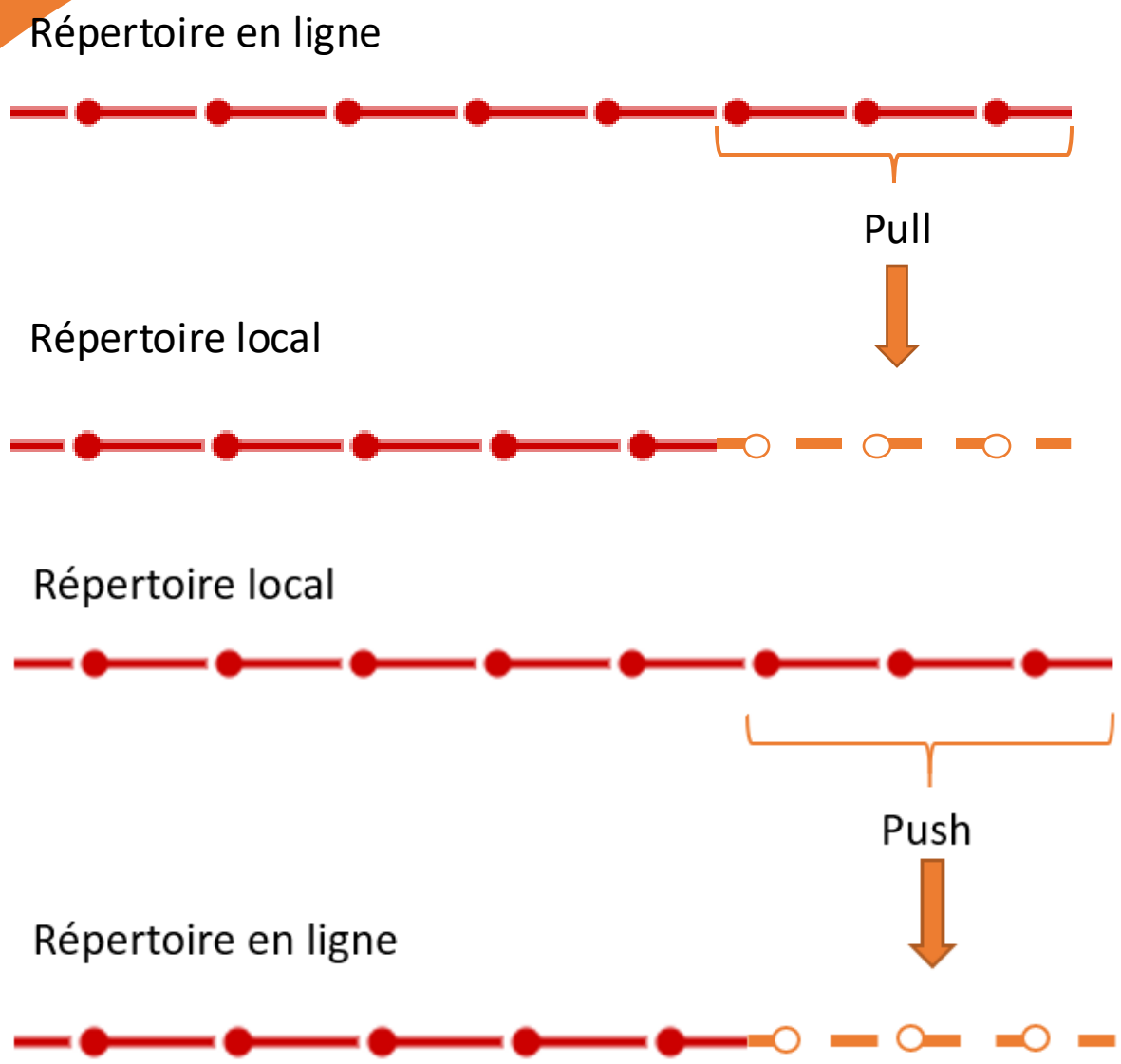
Commit

- Un commit est un point de sauvegarde du développement de code.
- Les commits permettent d'identifier des moments d'intérêt dans l'évolution du code.
- Ceux-ci permettent de faire des retours en arrière et d'annuler des changements spécifiques sans changer le restant du code.



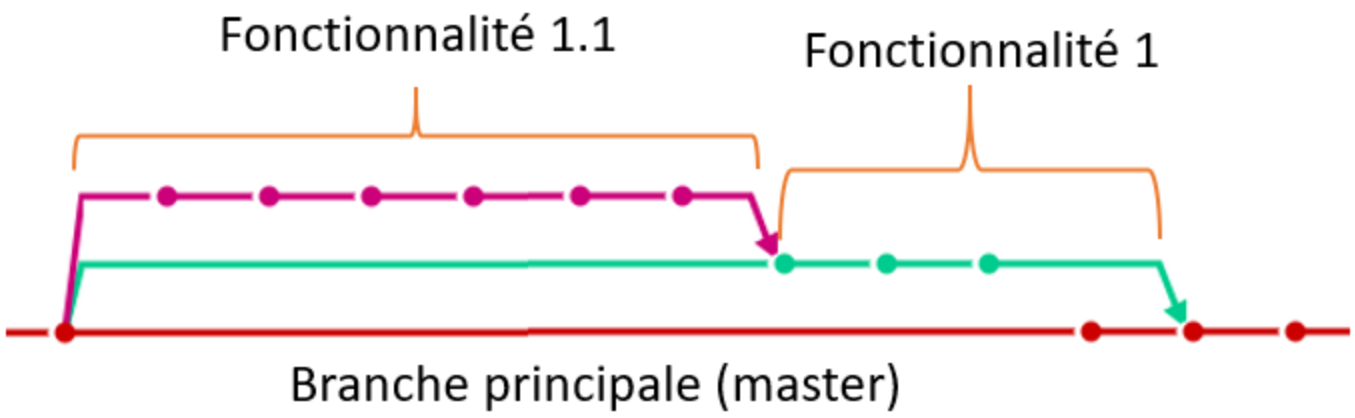
Push et pull

- Le push sert à envoyer les différents commits faits localement sur le répertoire en ligne.
- Le pull fait l'inverse, il permet d'ajouter localement les modifications qui sont présentes sur le répertoire en ligne.
- De manière générale, le push et le pull servent à mettre à jour le répertoire qui a du retard.



Branch

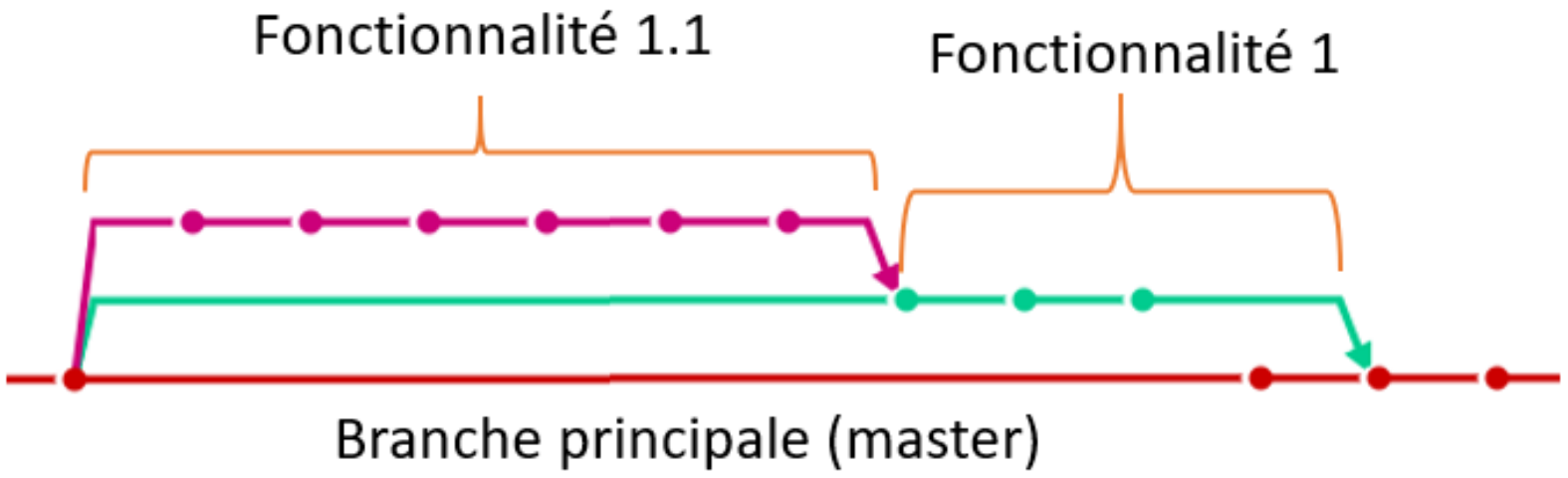
- Les branches sont des lignes parallèles de développements.
- Elles permettent de conserver une branche principale ayant constamment une version fonctionnelle du code.
- Les branches permettent aussi de diviser les tâches de développement de manière efficace.



Ici, on ajoute une fonctionnalité 1 dénotée en vert. Avant d'embarquer dans le développement de la fonctionnalité 1, puisqu'une partie de la fonctionnalité 1 était incertaine, une branche fonctionnalité 1.1, dénoté en mauve, a été créé afin de développer la partie incertaine de manière indépendante à l'entièreté de la fonctionnalité. Lorsque le développement d'une branche est terminé et fonctionnel, il est possible de l'inclure dans une autre branche.

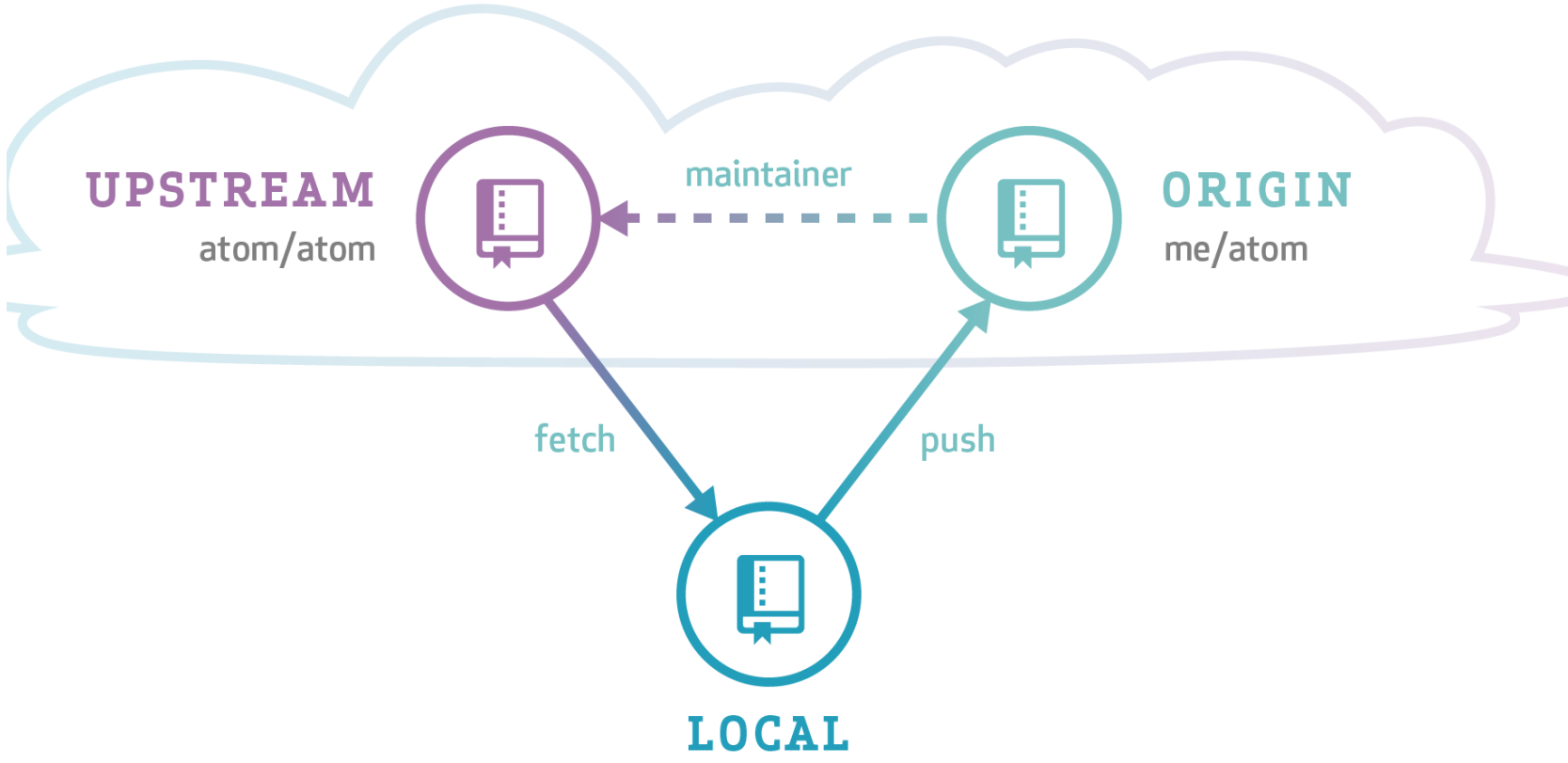
Merge

- Une fois que le travail sur une branche est terminé, il faut fusionner la branche terminée à une autre branche (étant très souvent master).
- Un merge de branches consiste à imbriquer les différentes modifications présentes dans les deux branches.
- Lors d'un merge, il peut y avoir des conflits. Dans d'autres mots, certaines modifications peuvent se contredire d'une branche à l'autre. Pour éviter ces conflits, il est important que les différences concernent différentes parties du code.



Fork

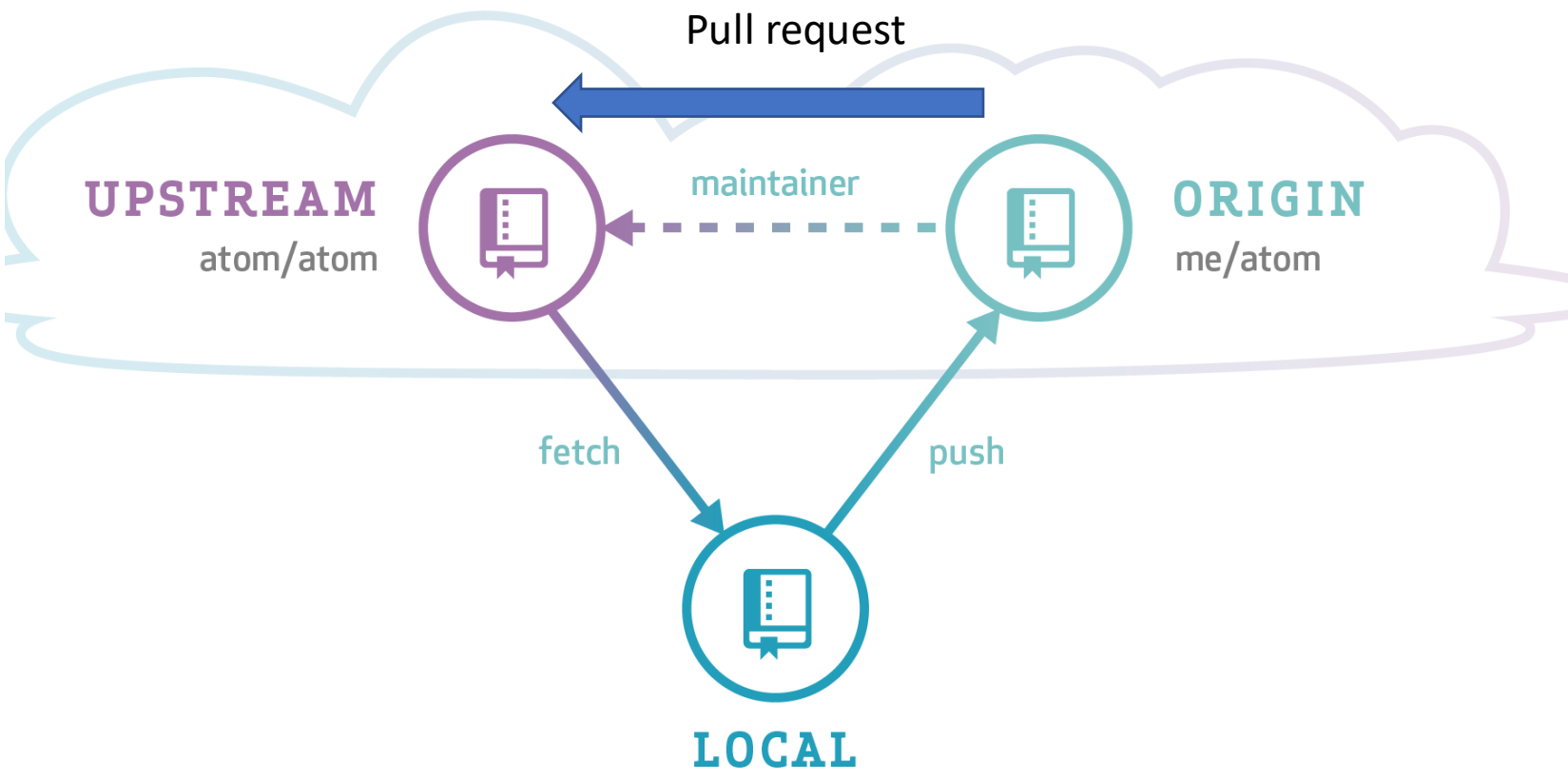
- Le fork consiste à crer un nouveau répertoire à partir d'un repertoire existant. Ici, le projet atom de l'utilisateur atom a été fork sous le même nom, mais cette fois de l'utilisateur «Me».
- Tous les push effectués sur me/atom n'affectent pas ce qui se trouve sur le répertoire original atom/atom.
- Si le répertoire atom/atom change et que nous voulons avoir les changements dans le répertoire me/atom, il faut faire un fetch (équivalent de pull, mais vers le répertoire atom/atom) et ensuite push vers me/atom.



git - How to fork a new branch from a repo when you already have the master? Stack Overflow. Accessed March 1, 2022.
https://stackoverflow.com/questions/36118433/how-to-fork-a-new-branch-from-a-repo-when-you-already-have-the-master

Pull request (PR)

- La pull request est l'équivalent de la merge request pour les branches. Cette fois, l'objectif est de fusionner l'ensemble d'un répertoire!
- Si la pull request est acceptée, les branches master, x et y du répertoire me/origin vont être joint au répertoire atom/atom. Cela implique que la branche master sera mise à jour et les branches x et y seront ajouté au répertoire atom/atom.



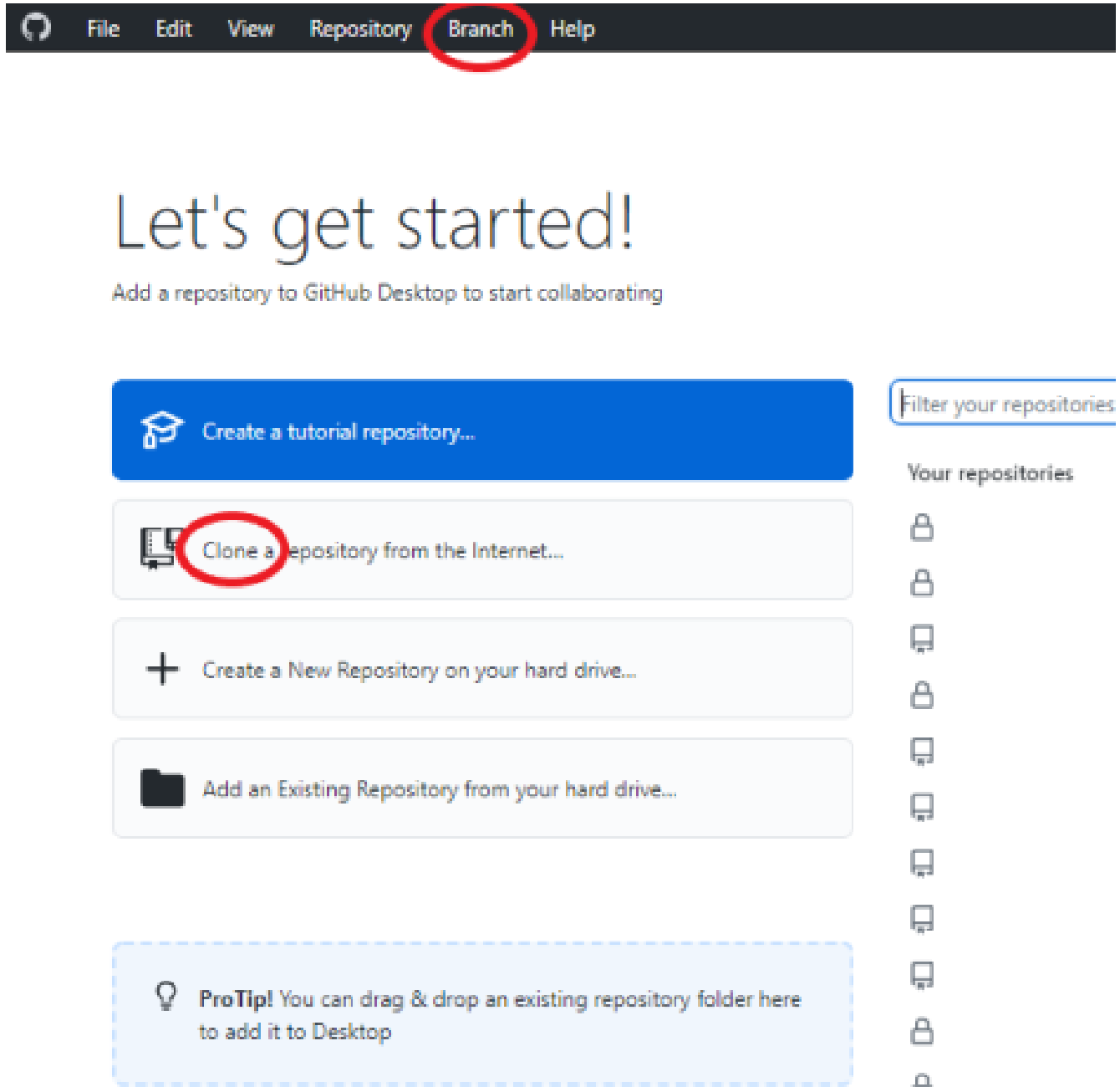
git - How to fork a new branch from a repo when you already have the master? Stack Overflow. Accessed March 1, 2022. https://stackoverflow.com/questions/36118433/how-to-fork-a-new-branch-from-a-repo-when-you-already-have-the-master

Exemples d’outils pour travailler avec Git sur votre ordinateur

- Exemple avec Github desktop
- Exemple dans un environnement de développement (Pycharm)
- Exemple rapide dans un terminal

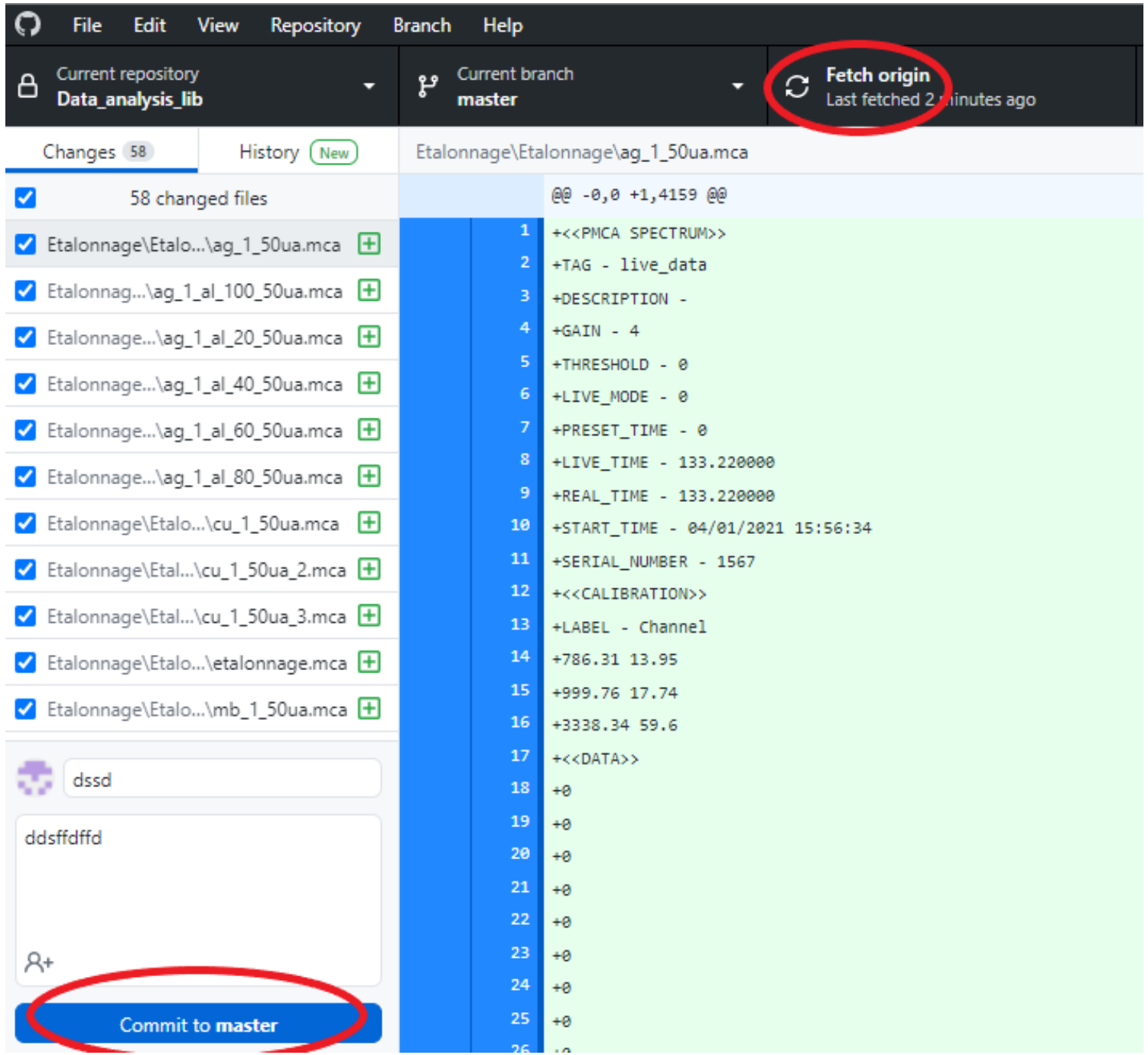
Github Desktop

- Github Desktop est une interface graphique permettant d'appeler les différentes commandes git.
- Sur l'image ici, on peut apercevoir l'option permettant de cloner un répertoire se trouvant sur un répertoire étant sur un serveur quelconque (Github, Gitlab, etc).
- Il est aussi possible de voir l'option discrètement placée en haut.



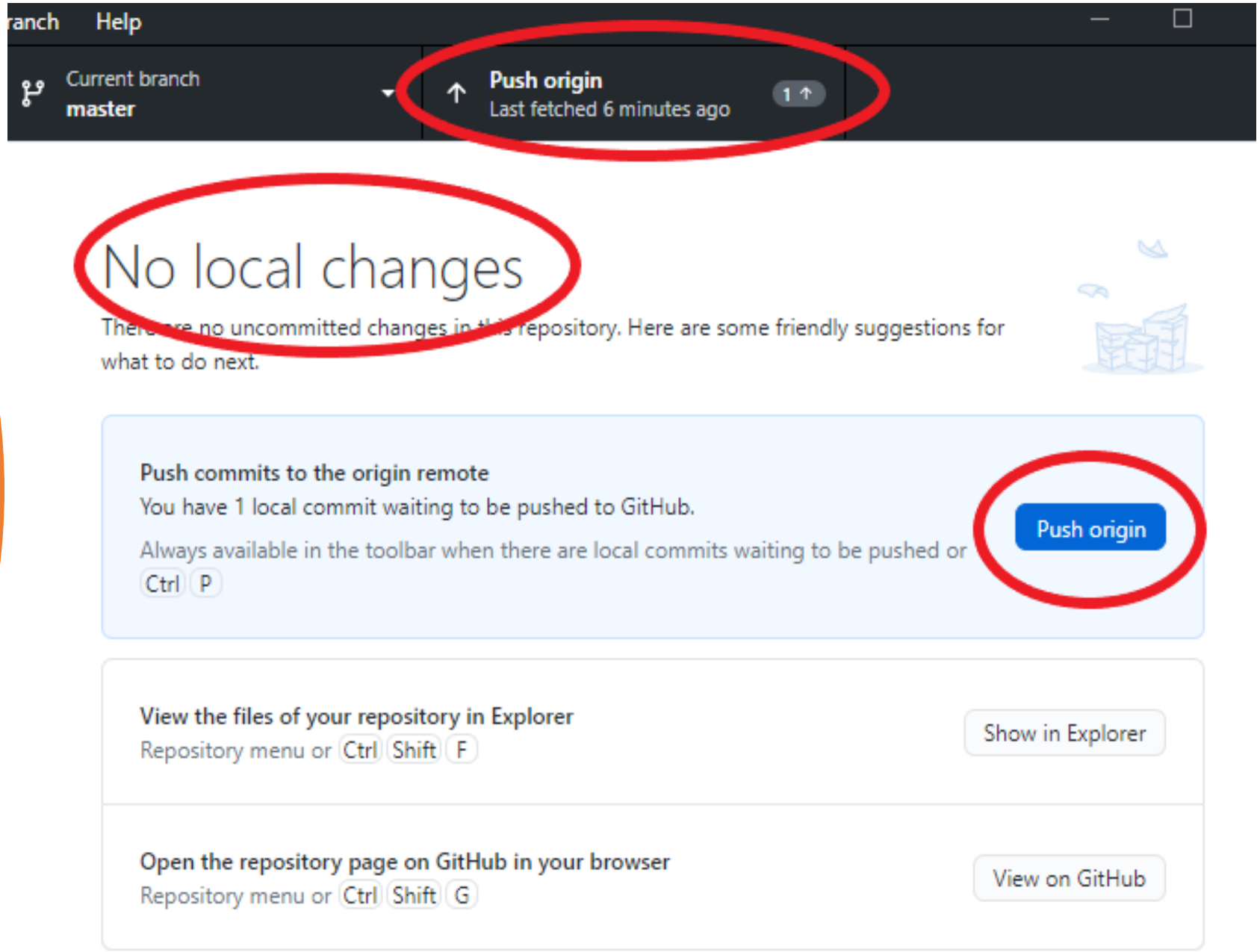
Github Desktop

- Ici, on peut voir l'option Fetch (qui deviendra pull lorsque Github Desktop détecte des mises à jour sur le répertoire du serveur)
- Dans la partie avec des crochets, il s'agit de toutes les modifications qui seront incluses dans le commit.
- Dans la section commit du bas, il y a l'emplacement pour faire le message et ajouter une description plus détaillée



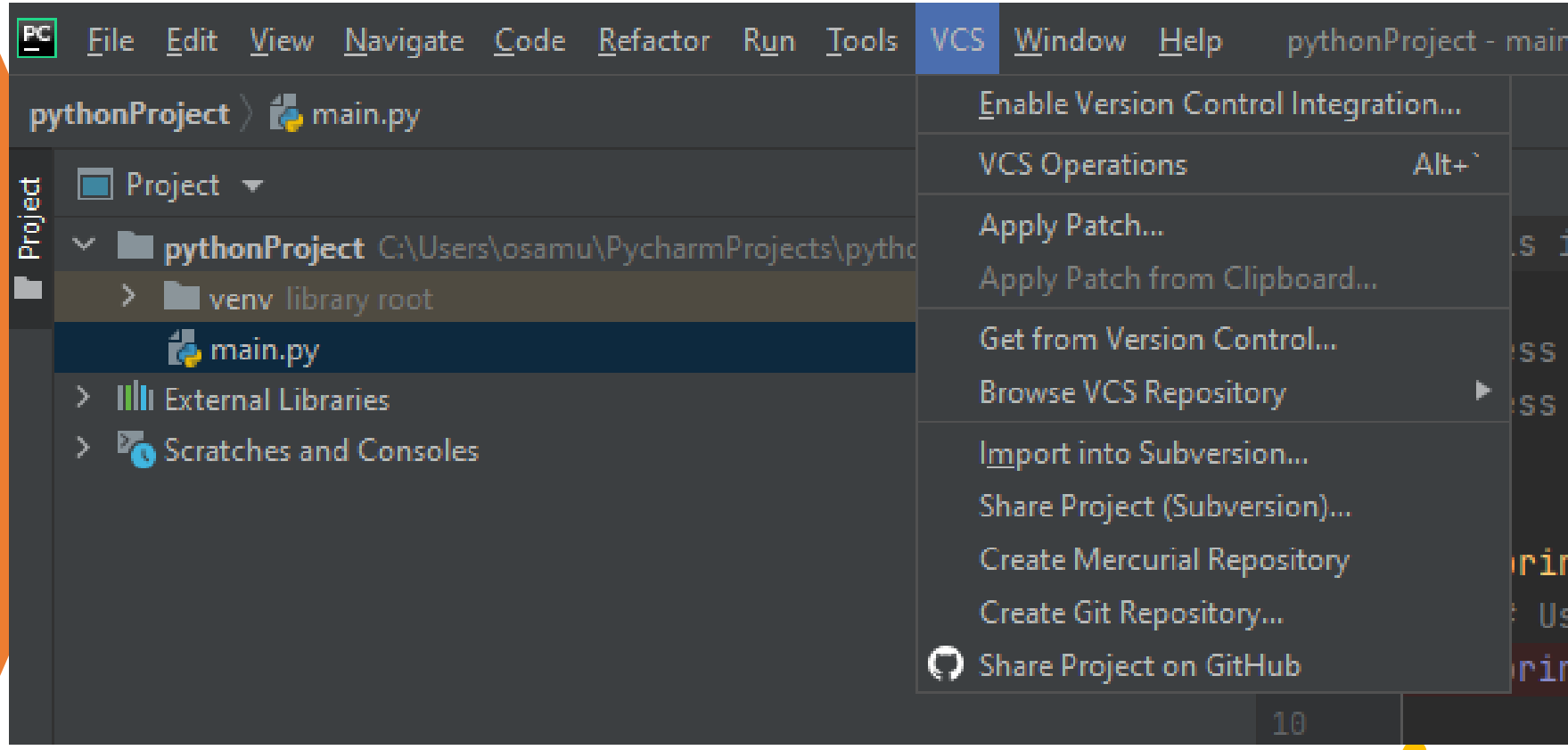
Github Desktop

- Après avoir effectué le commit, il est possible de voir qu'aucun changement restant n'est présent. Il reste simplement à les push vers le répertoire du serveur.
- Il est pertinent de remarquer la branche sur laquelle le push sera effectué, ici c'est sur master. Dans une situation où il ne s'agit pas de la branche cible, il aurait fallu changer la branche avant le commit.



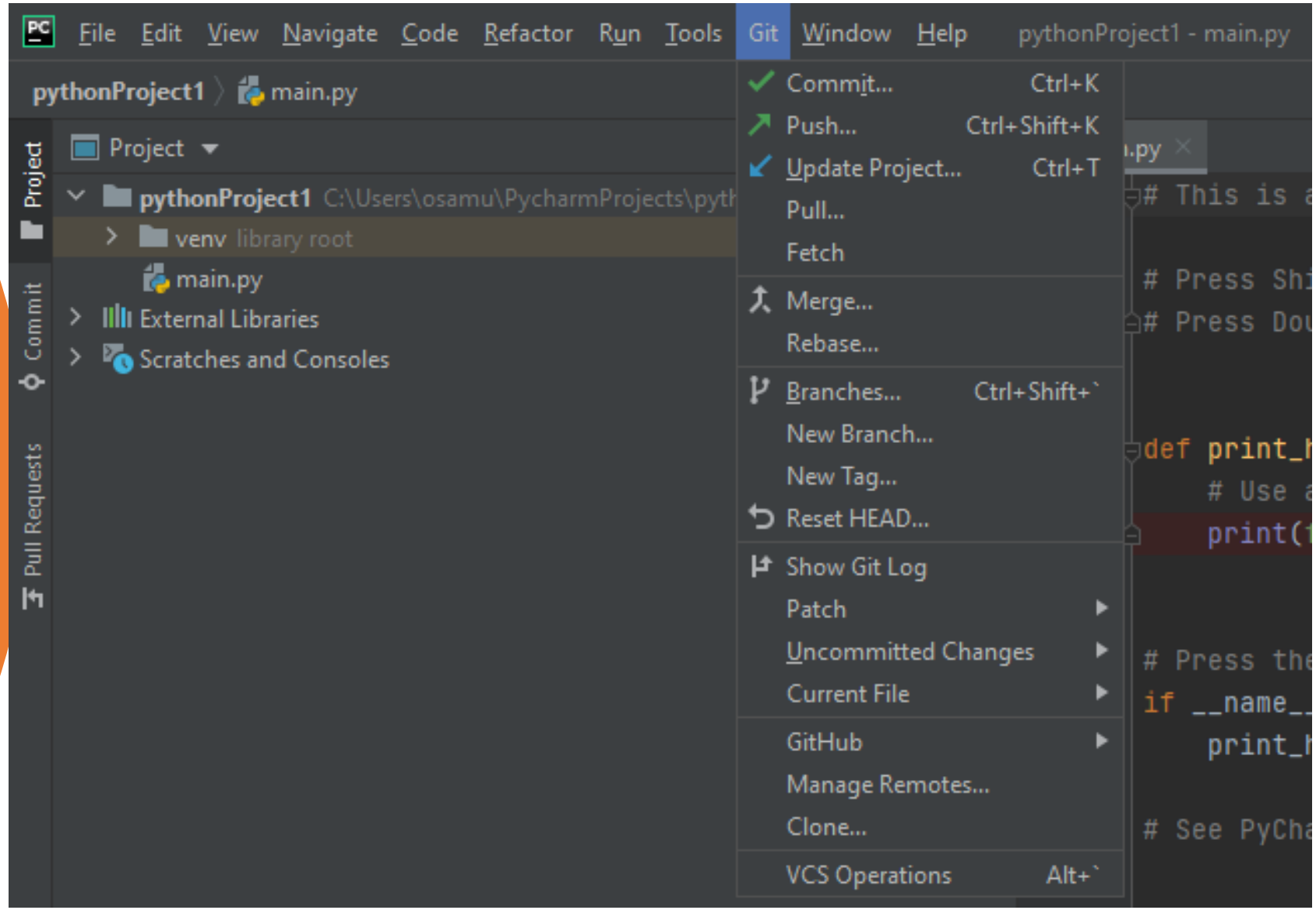
Dans un IDE

- Les IDE possèdent souvent des raccourcis git pour faciliter la gestion de versions.
- Il s'agit de l'interface dans Pycharm lorsque le projet n'est pas relié à git. Ici l'option share on github permettrait de créer un répertoire sur github et le relié à ce projet existent. L'option create git repo ne crée que le répertoire git local, mais pas celui sur un serveur!!



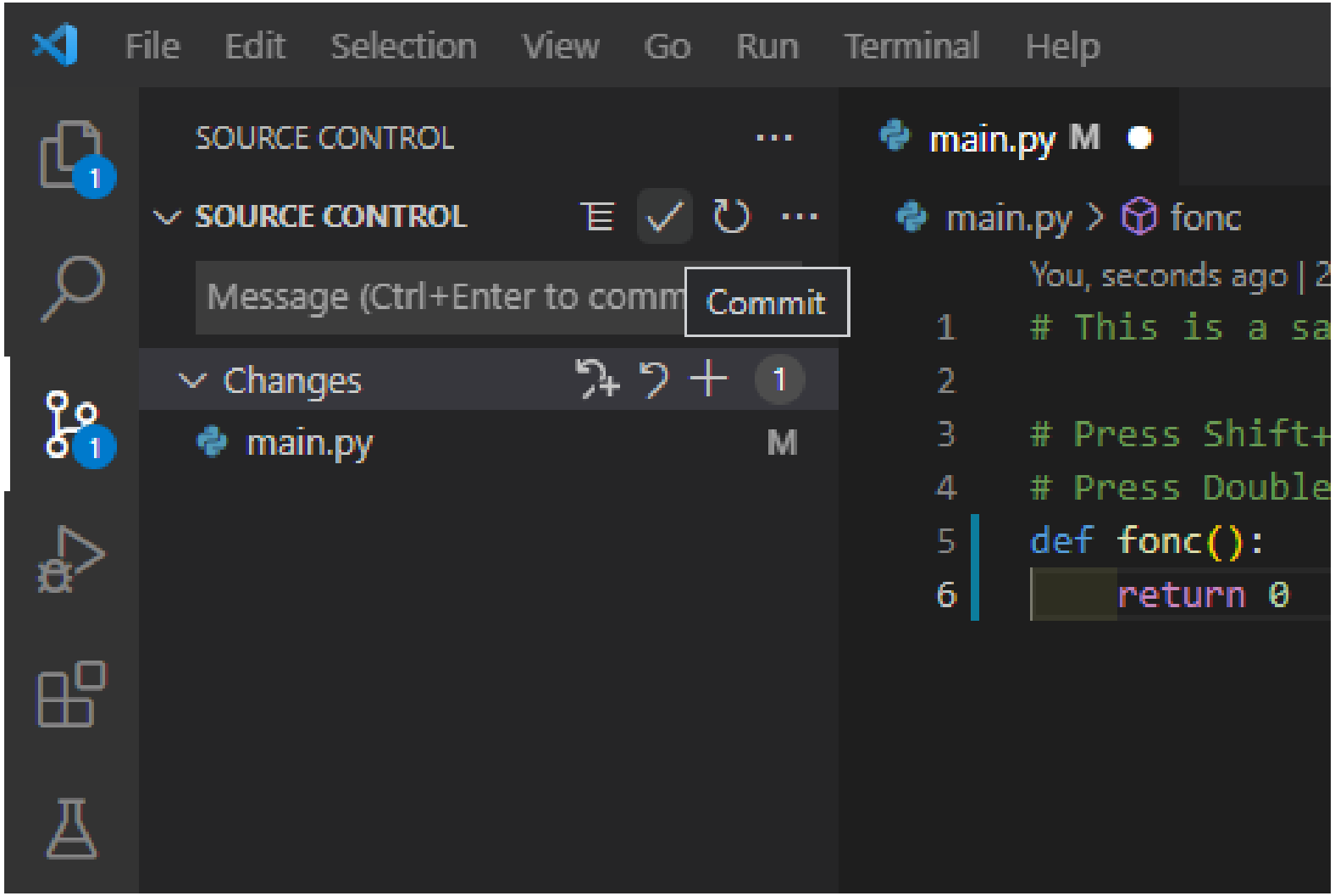
Pycharm

- Lorsque le projet fait partie d'un répertoire git local, les options de Pycharm deviendront celles affichées ici. Il est possible de remarquer une bonne partie des commandes git tel que le commit, le pull (update project), le push, le merge et même les branches!!
- L'important est de toujours repérer les mots clés!



Visual Studio Code

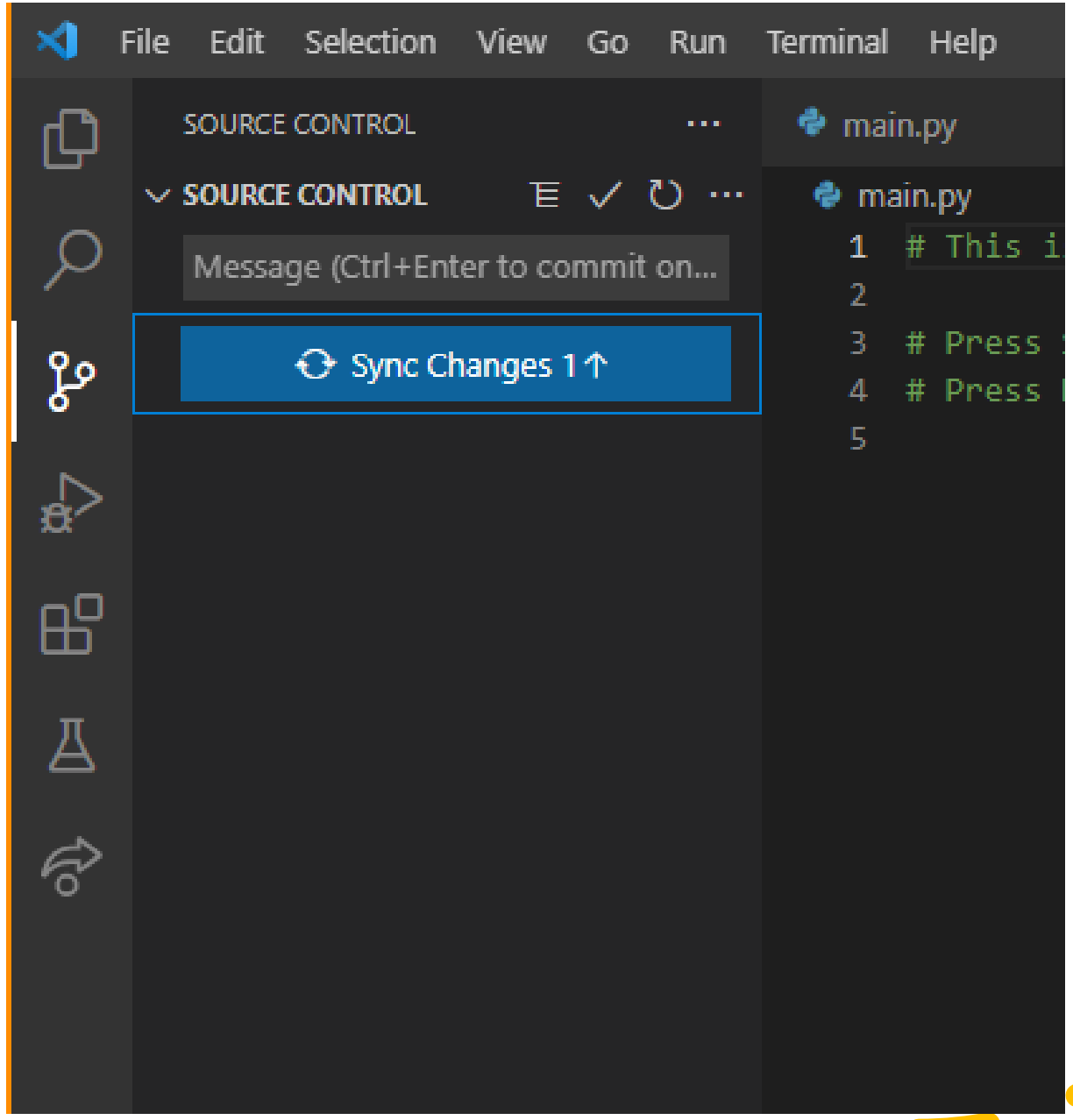
- Voici l'équivalent dans Visual Studio. Ici, le crochet effectue le commit et il est possible de voir les changements effectués dans le code.
  L'Interface est moins claire, mais les outils sont présents!



Visual Studio Code

- Après le commit nous avons l'équivalent du push (Sync changes)

- Pour les opérations plus complexes, je recommande Github Desktop, Pycharm ou le Terminal!



Par le terminal

- Ici nous avons l'outil ultime! Git a originalement été conçu pour être utilisé via le terminal. Cela implique que l'entièreté des commandes peut se faire à partir du Terminal.
- Dans le cas d'opérations très particulières (souvent, cause par des problèmes particuliers), seul le terminal pourra vous aider. Ça vaut éventuellement la peine d'apprendre à l'utiliser!

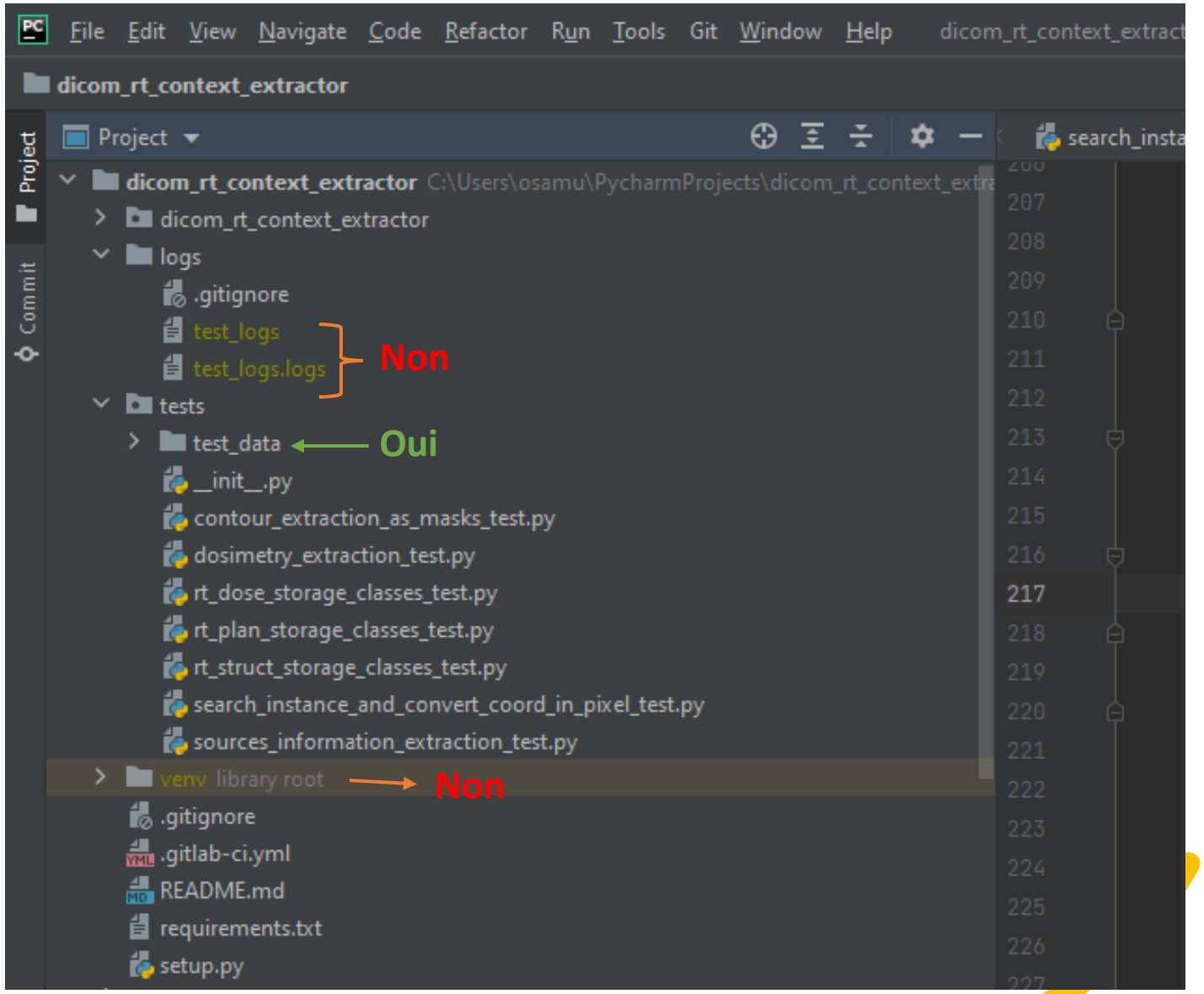
```
C:\Users\osamu>git clone https://github.com/smichi23/py3ddose.git
Cloning into 'py3ddose'...
remote: Enumerating objects: 42, done.
remote: Counting objects: 100% (10/10), done.
remote: Compressing objects: 100% (7/7), done.
remote: Total 42 (delta 1), reused 9 (delta 1), pack-reused 32
Unpacking objects: 100% (42/42), done.

C:\Users\osamu>cd py3ddose

C:\Users\osamu\py3ddose>git add *

C:\Users\osamu\py3ddose>git commit -m "adding spaces in README"
[master e75205c] adding spaces in README
 1 file changed, 1 insertion(+), 1 deletion(-)

C:\Users\osamu\py3ddose>git push
```

Comment utiliser Git au quotidien

- Qu’est-ce qui ne devrait pas se retrouver sur un serveur Git
- Cycle typique de développement
- Révision de code

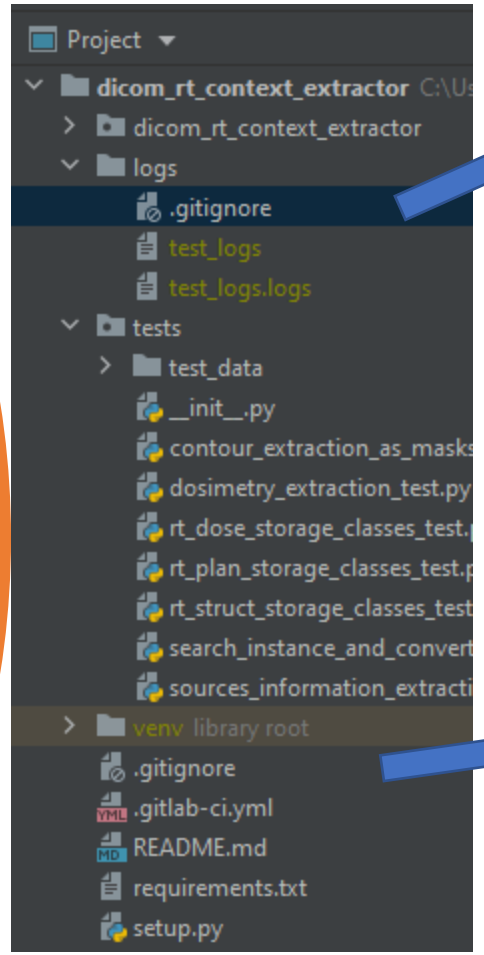
Qu’est-ce qui ne devrait pas se retrouver sur un serveur Git?

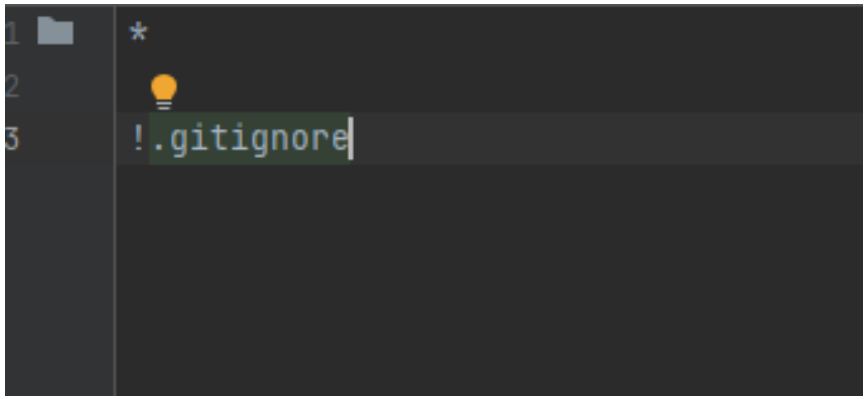
- Tout ce qui n’est pas nécessaire pour exécuter le code et tout ce qui ne fait pas partie de la documentation. Il faut garder que ce qui est essentiel afin de ne pas polluer le répertoire afin que la structure du répertoire soit claire pour tout éventuel utilisateur!

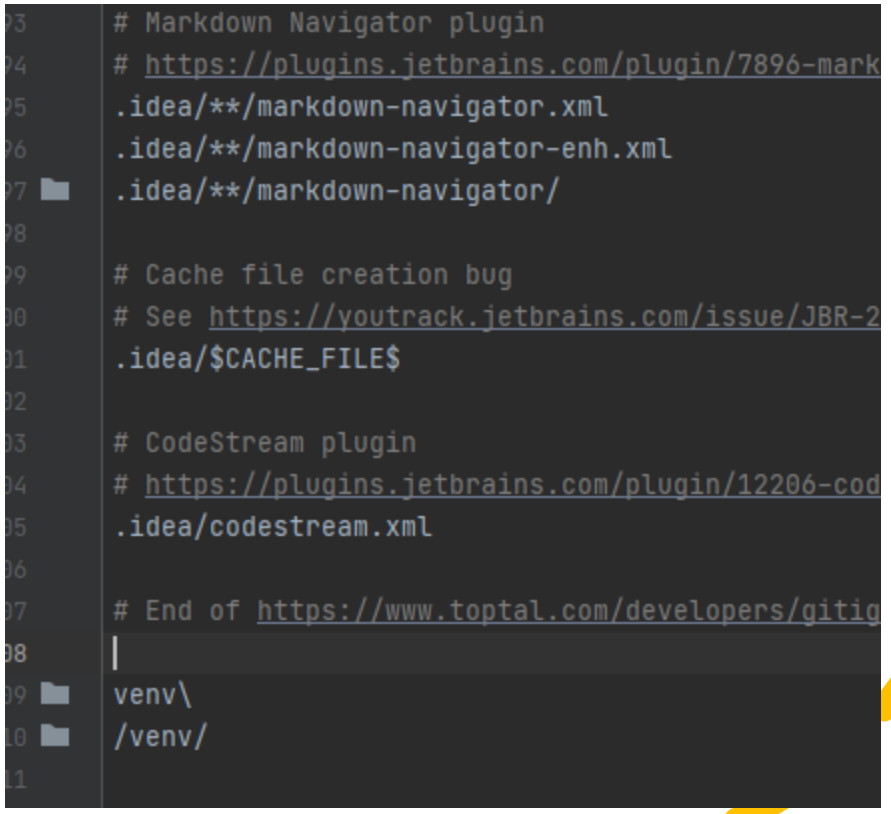


Le .gitignore

- Le .gitignore sert à spécifier à git ce qu'il doit ignorer. Ici, le .gitignore spécifie de ne pas suivre les changements apportés et mettre sur le serveur l'environnement virtuel (venv). Puisque le dossier logs doit être conservé, mais pas ce qui est à l'intérieur, un second .gitignore est créé dans le dossier logs pour ignorer tout sauf le .gitignore
- https://www.toptal.com/developers/gitignore
- Ce site génère des .gitignore.

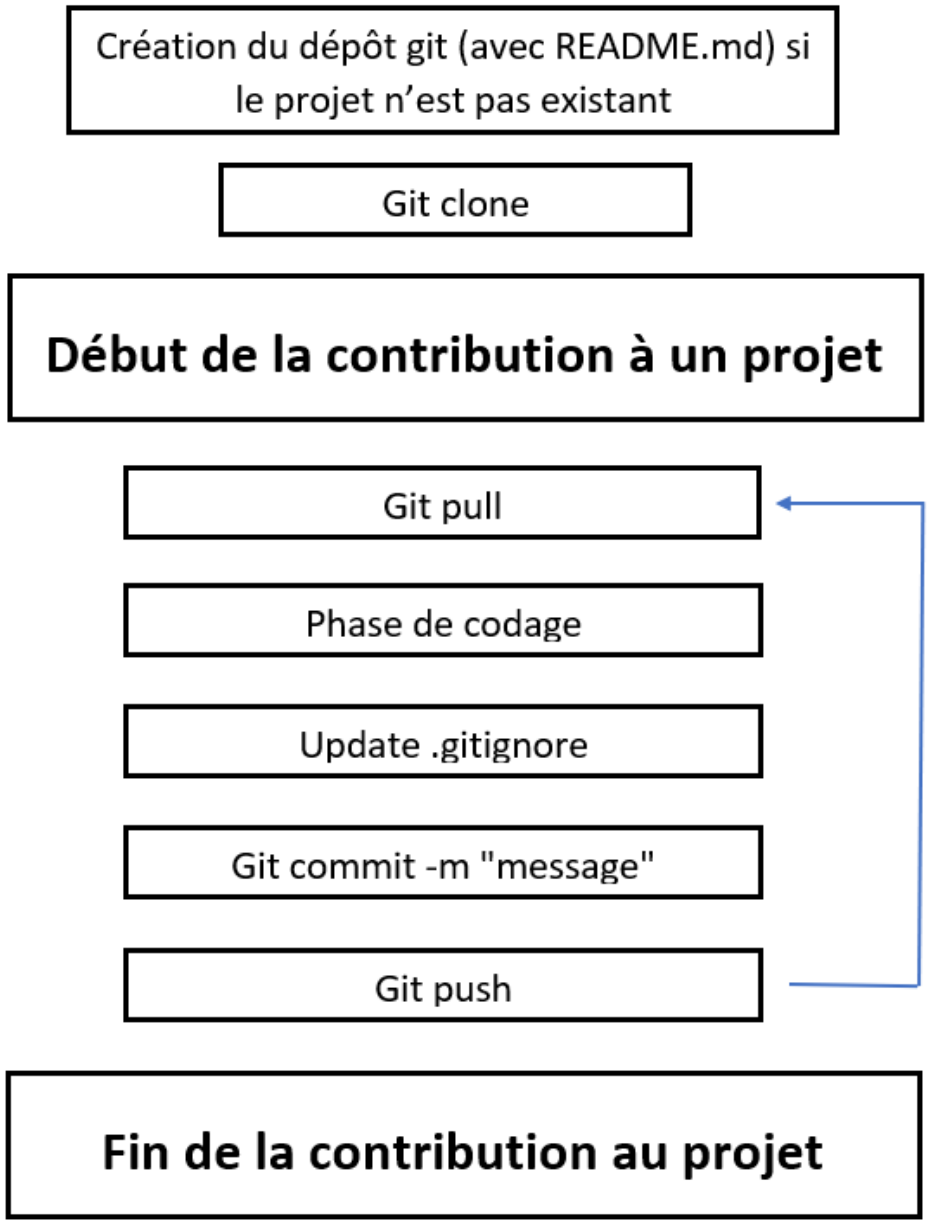






Cycle typique de développement

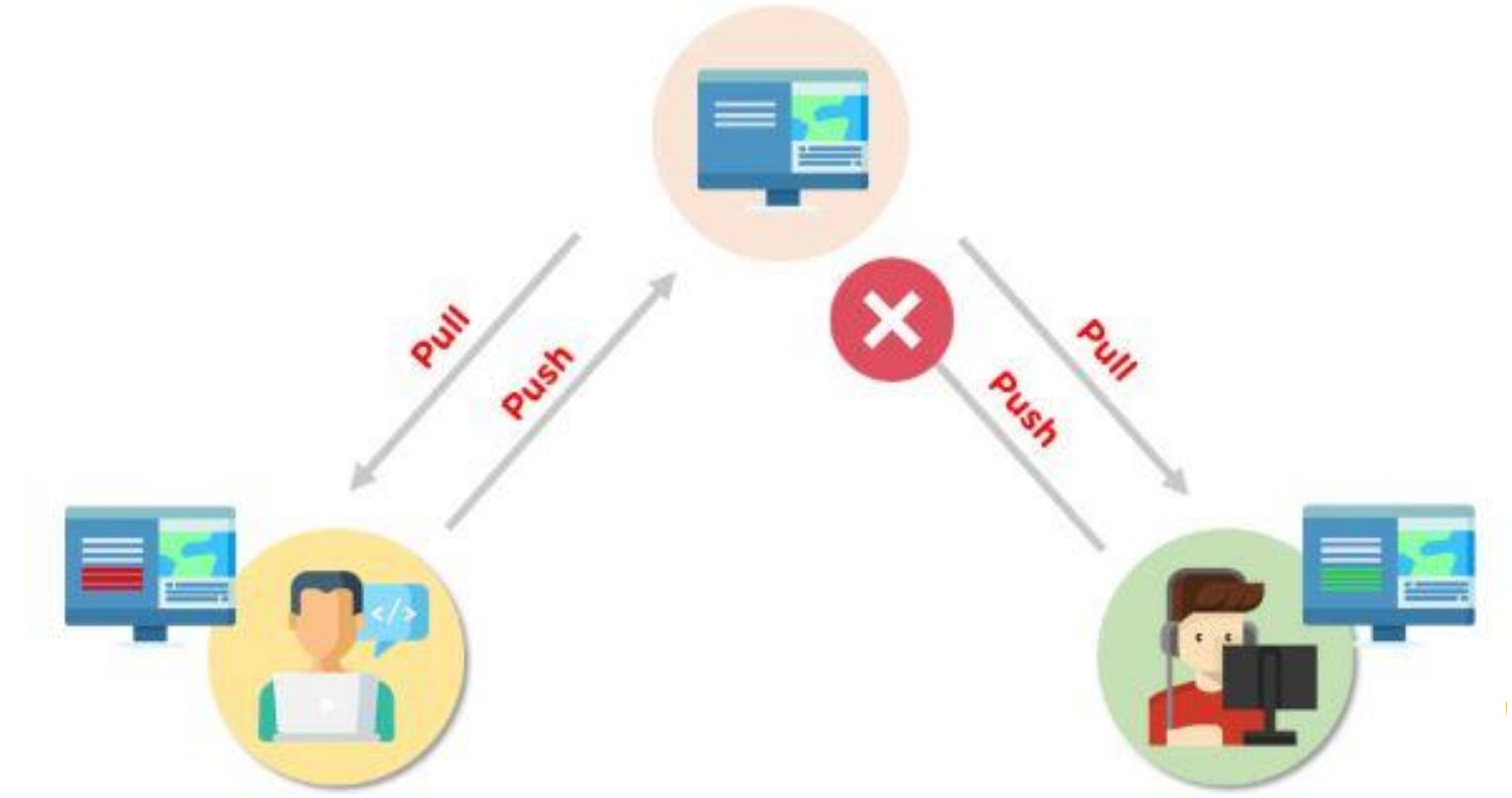
- Considérant une utilisation minimaliste de git, voici à quoi devrait ressembler le cycle de développement de code
- Il est important de noter que de travailler directement sur la branche principale comporte des inconvénients et des risques!
- Il est important d'avoir un bon rythme de commits et des messages significatifs!



À chaque fois qu'on developpe du code, on recommence le cycle en bleu.

Attention aux conflits!

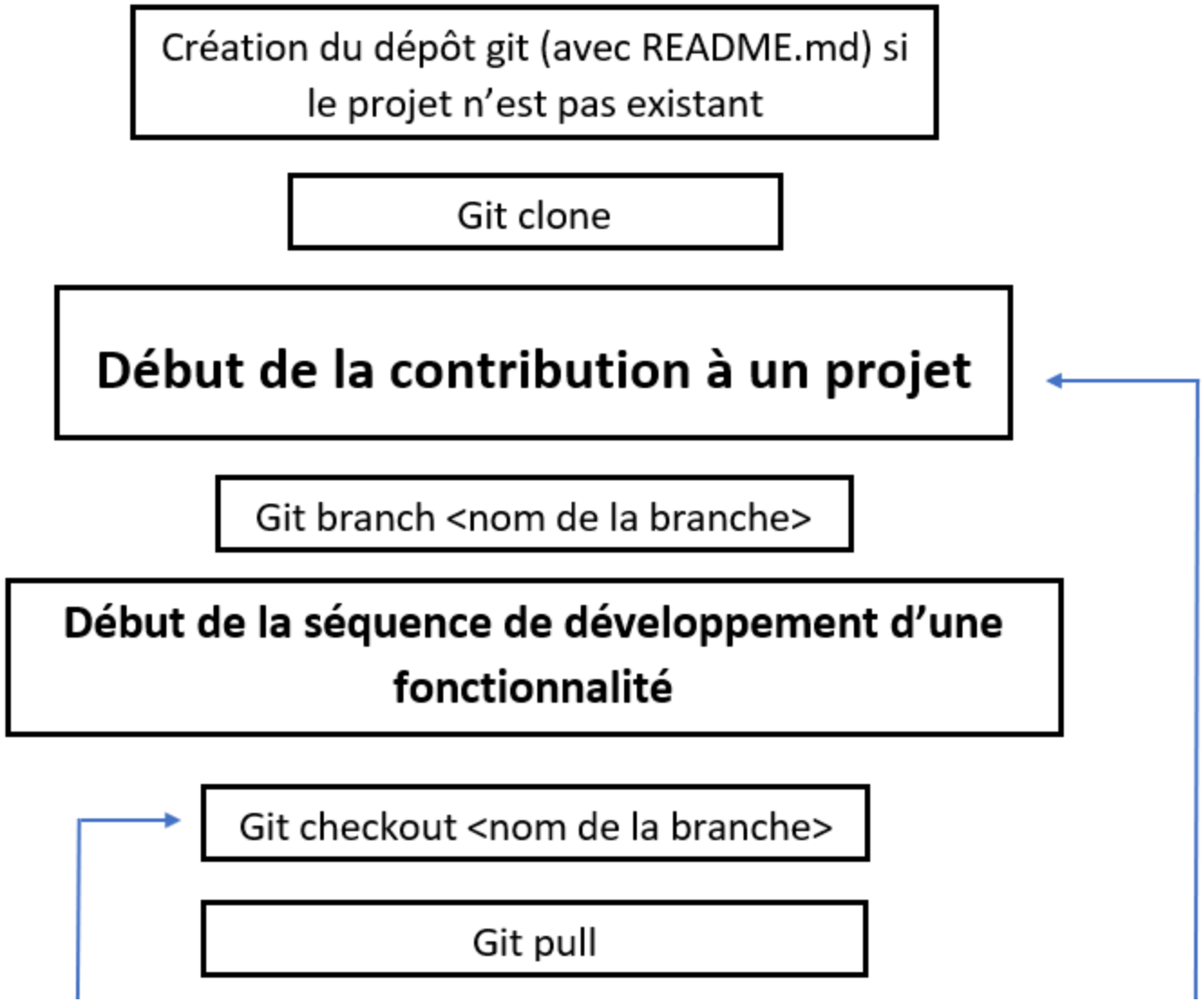
- Il faut s’assurer de pull avant de modifier le code. Car si un collègue effectue des modifications et qu’un push contredit ce qui n’a pas été pull, cela crée un conflit.
- Pour règler les conflits, il faut manuellement choisir quelles modifications conserver et lesquelles à ignorer.
- Essentiellement, il ne faut jamais modifier la même ligne de code lorsqu’on collabore



How to Resolve Merge Conflicts in Git? Simplilearn [Updated]. Simplilearn.com. Accessed March 2, 2022. https://www.simplilearn.com/tutorials/git-tutorial/merge-conflicts-in-git

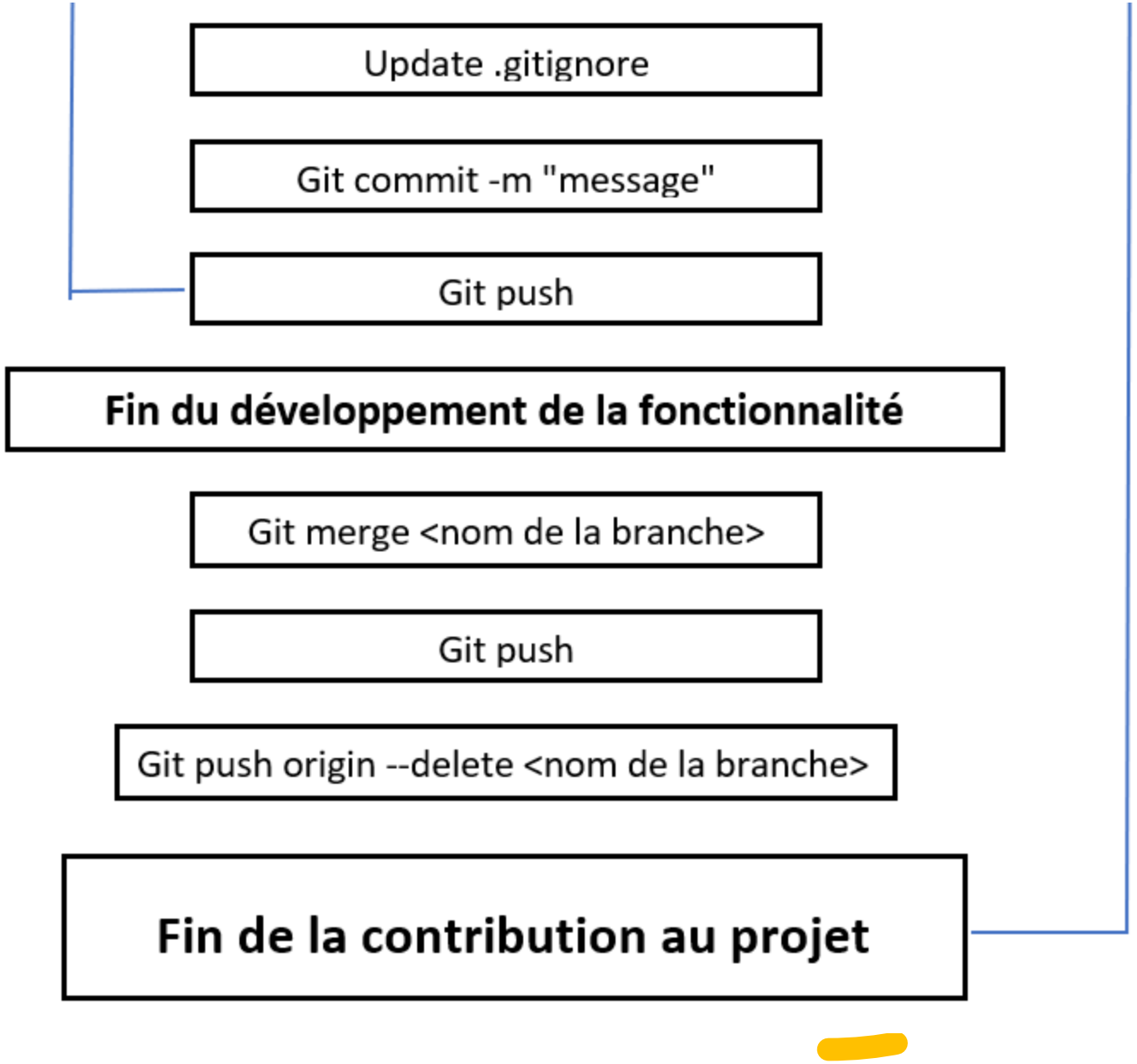
L’utilisation des branches

- Les étapes à ajouter sont simplement les étapes pour créer une branche et s’assurer de travailler sur la bonne branche en effectuant un checkout.
- Sinon le reste ne change pratiquement pas!



L’utilisation des branches

- À la fin du développement de la nouvelle fonctionnalité, if faut fusionner la branche de développement dans la branche du code utilisable.
- Il ne faut pas oublier de push la fusion, puisque pour le moment, celle-ci n’est que locale!



Révision de code

- Lorsqu'on veut merge une branche dans une autre, plutôt que de le faire local, il est souvent possible de faire une requête pour merge les branches directement sur le serveur.
- Les requêtes pour merge deux branches doivent être acceptées manuellement. Avant d'être acceptée, la requête offre une excellente opportunité pour reviser le code de ses collègues.

```
+    def get_query_answer_number_of_results(self, query_identifier: str,
+            params: Dict = None,
+            **kwargs) -> list:
+
+        """Get all content of specified answer of C-Find
+
+            Parameters
+            ----------
+            query_identifier
+                Query identifier.
+            params
```

· 1 year ago Maintainer

Quand les `params` ne sont pas nécessaire, je ne fait qu'enlever le paramètre (C'est nécessaire ici?)

Samuel Ouellet @sam23 · 1 year ago Developer

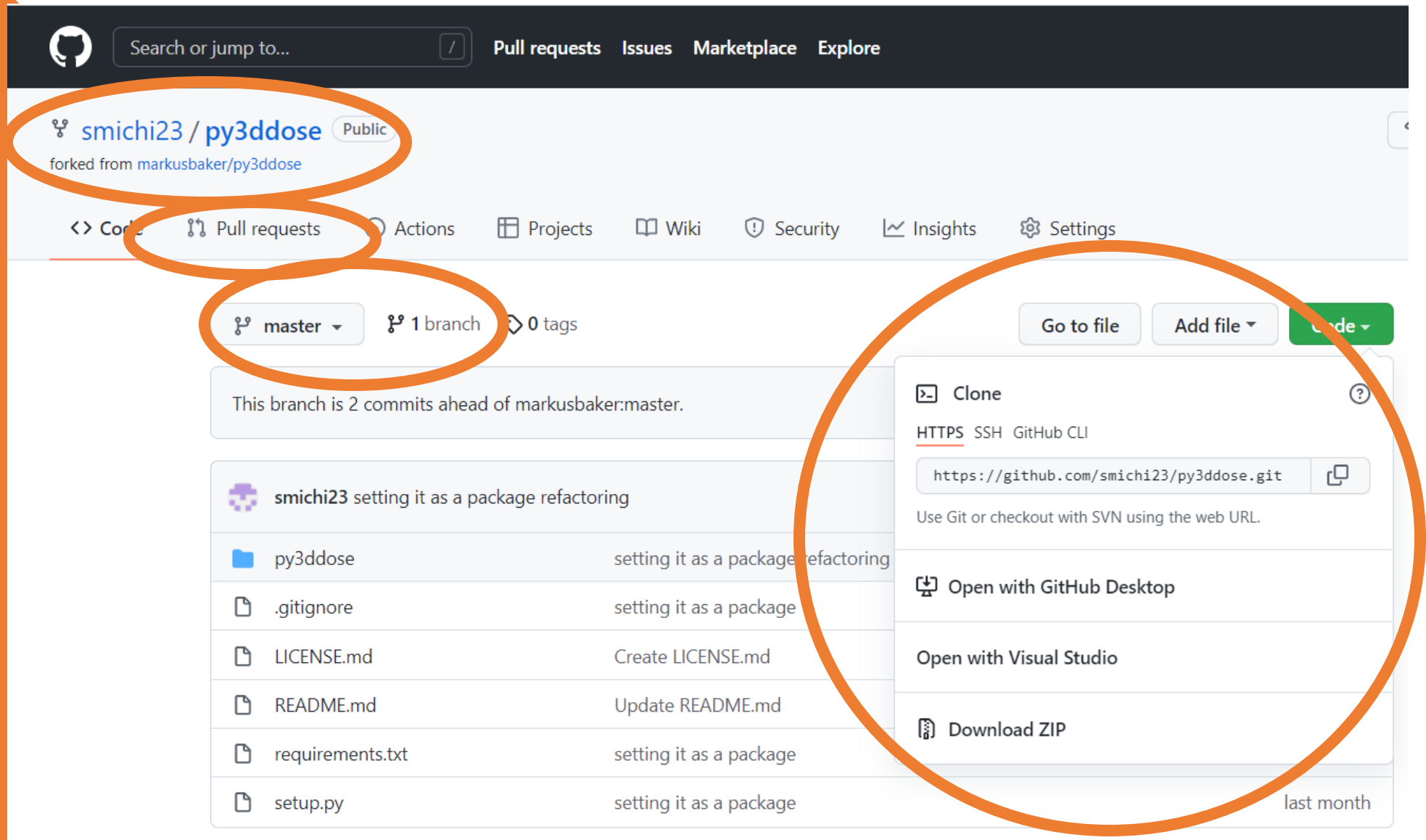
Non ils ne sont pas nécessaire

Qu’est-ce qui se trouve sur un serveur Git?

- Exemple de navigation sur Github
- Exemple de navigation sur Gitlab

Github

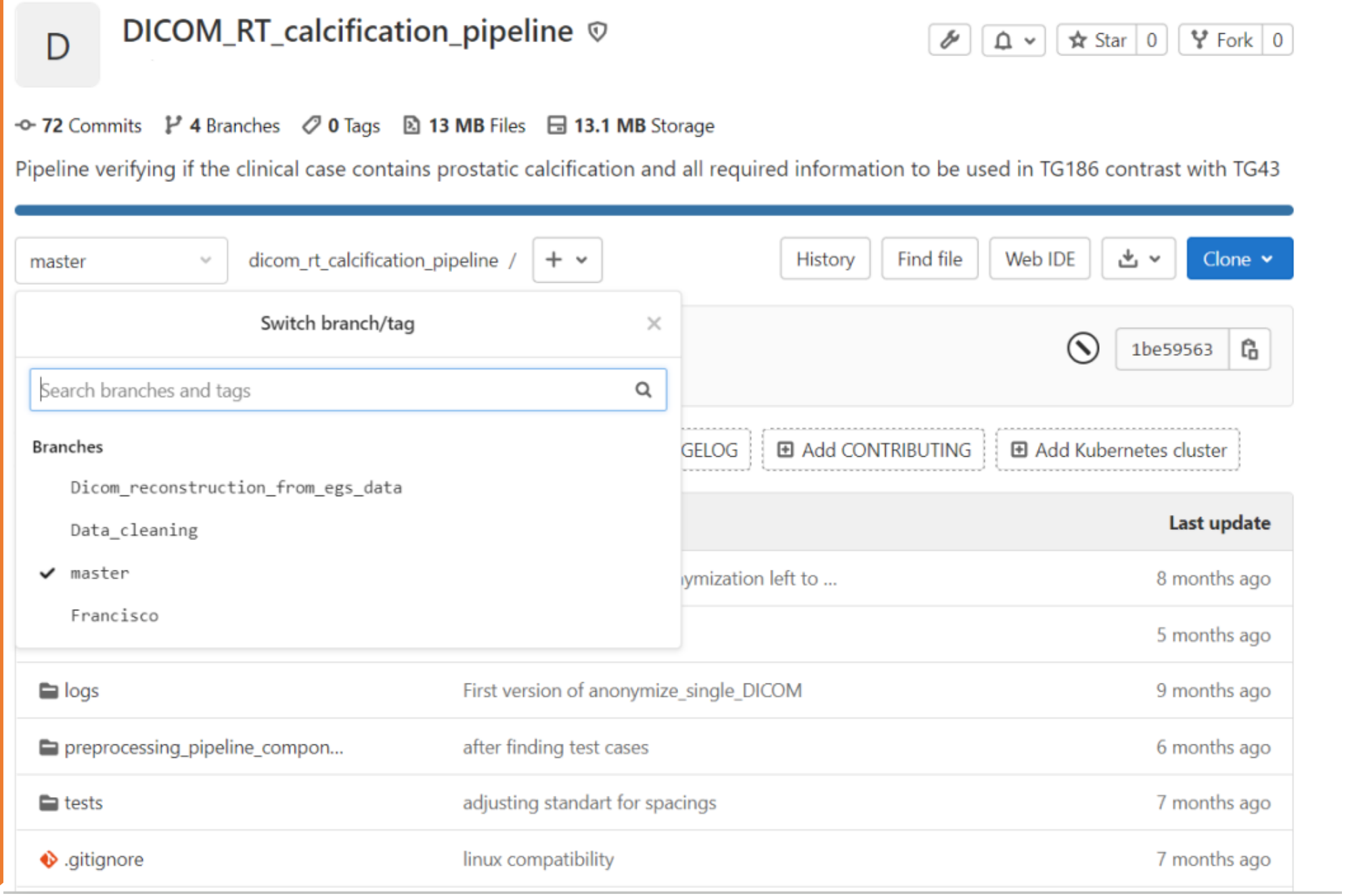
- En plus de voir le répertoire, il est possible d'obtenir bien plus d'information sur les serveurs Git!
- Ici, on peut voir quel répertoire a été utilisé pour faire le fork, on peut voir l'onglet pull request, on peut voir les différentes branches (ici, il y en a juste une) et on peut voir comment cloner le code.



Ouellet S. *Smichi23/Py3ddose*.; 2022. Accessed March 2, 2022. https://github.com/smichi23/py3ddose

Gitlab

- Ici c'est encore la même chose, on peut voir le clone, les branches les commits
- La majorité des serveurs Git vont offrir l'information de base. Github et Gitlab offrent beaucoup plus que les informations de bases!

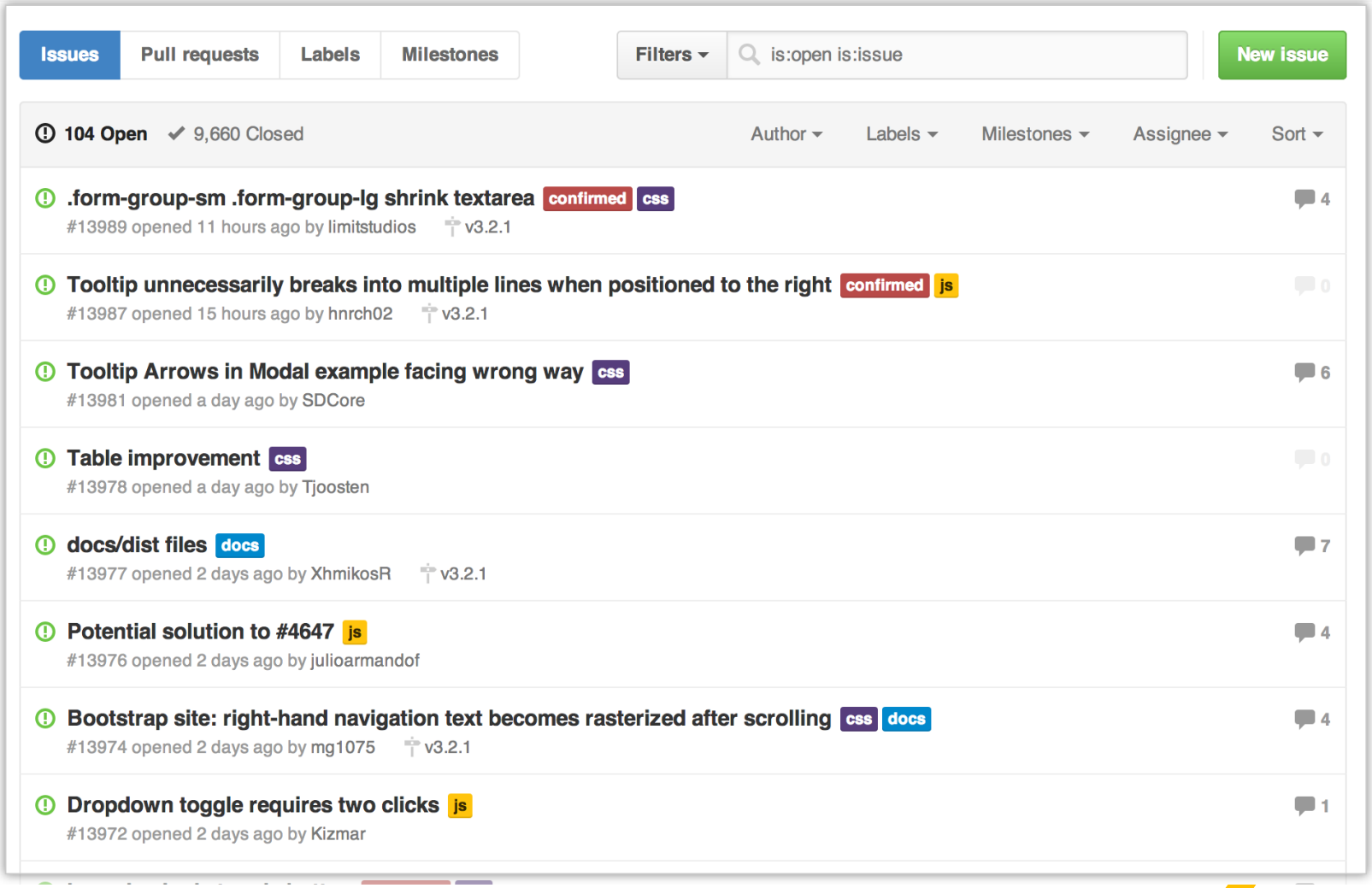


Introduction aux outils qu'offrent les serveurs Git pour les développeurs

- Le système d'issues pour la réparation de bogues
- Intégration en continu

Le système d'issues pour la réparation de bogues

- En plus de permettre le développement collaboratif dans un même répertoire, la majorité des serveurs Git permettent de gérer la répartition des tâches pour la résolution de problèmes d'utilisation.
- Parmi la liste d'issues, le programmeur se choisit un problème à régler, se créer une branche dédiée pour ensuite contribuer à l'avancement du projet sans avoir besoin d'être géré.



Mastering Issues · GitHub Guides. (s. d.). Consulté 17 mai 2021, à l'adresse https://guides.github.com/features/issues/

Intégration en continu

- L'intégration en continu ajoute une couche de sécurité à la modification du code.
- Avant qu'un push soit accepté, il est possible d'exécuter des scripts afin de vérifier que les modifications proposées réussissent les tests unitaires.
- Cela permet de s'assurer de ne jamais push des erreurs majeures à son insu.

```
before_script:
  - /opt/rh/rh-python36/root/usr/bin/pip3.6 install virtualenv
  - virtualenv -p /opt/rh/rh-python36/root/usr/bin/python3.6 venv
  - ./venv/bin/pip install --upgrade pip
  - ./venv/bin/pip install -r requirements.txt


stages:
  - test

test:
  stage: test
  script:
    - ./venv/bin/python -m unittest discover -s tests -p *test.py
```

Pipeline #661 passed with stage in 17 seconds